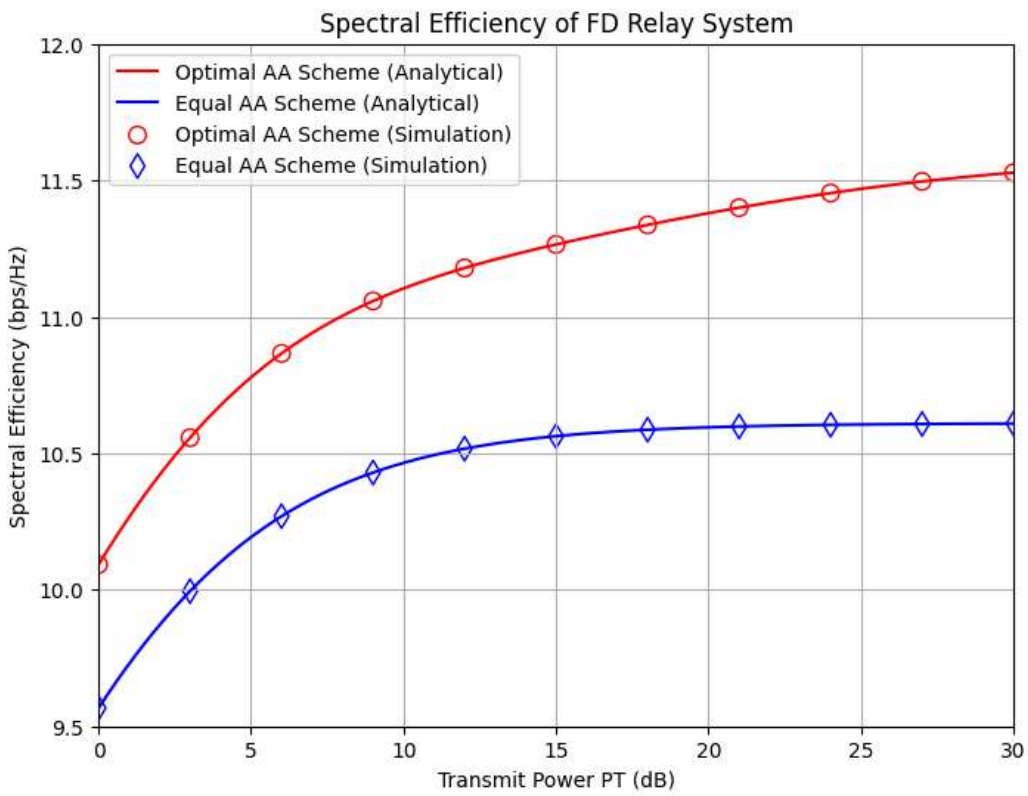
Reprodução da Simulação do código que consta no artigo [1] usado como referência para a realização do trabalho.

```
import numpy as np  # Usado para operações matemáticas e matriciais
import matplotlib.pyplot as plt  # Usado para plotar gráficos

# Definição de parâmetros fixos do sistema
NR = 20  # Número de antenas no relay
PdB_full = np.linspace(0, 30, 200)
PdB_sim = np.linspace(0, 30, 11)
sigma2 = 10**(-20/10)
kappa = 0.01
rho = 0.2
nu = 2

# Listas para armazenar os resultados de eficiência espectral analítica
SE_optimal_analytical = []
SE_equal_analytical = []

# Calculando eficiência espectral analítica para cada valor de potência em PdB_full
for P_dB in PdB_full:
    Ps = Pr = 10**(P_dB/10)
    a1 = NR * Ps * (1 - rho)**(-nu)
    a2 = NR * Pr * rho**(-nu)
    b = kappa * Pr
    c1 = a1 - 2*a2*b - a2 + b
    c2 = a1 + b + 1
    eta_optimal = (c2 - np.sqrt(c2**2 - 4 * c1 * c2)) / (2 * c1)
    sinr_optimal_analytical = ((1-eta_optimal)*a1*eta_optimal*a2) / ((1-eta_optimal)*a1*sigma2 + (eta_optimal*a
    sinr_equal_analytical = ((1-0.5)*a1*0.5*a2) / ((1-0.5)*a1*sigma2 + (0.5*a2 + sigma2)*(b + sigma2))
    SE_optimal_analytical.append(np.log2(1 + sinr_optimal_analytical))
    SE_equal_analytical.append(np.log2(1 + sinr_equal_analytical))

# Simulação de Monte Carlo
SE_optimal_sim = []
SE_equal_sim = []

# Executando simulações para cada valor de potência em PdB_sim
for P_dB in PdB_sim:
    Ps = Pr = 10**(P_dB/10)
    sinr_optimal_samples = []
    sinr_equal_samples = []
    for _ in range(100000):
        h_SR = np.sqrt(0.5) * (np.random.randn() + 1j * np.random.randn())
        h_RD = np.sqrt(0.5) * (np.random.randn() + 1j * np.random.randn())
        a1 = NR * Ps * (1 - rho)**(-nu)
        a2 = NR * Pr * rho**(-nu)
        b = kappa * Pr
        c1 = a1 - 2*a2*b - a2 + b
        c2 = a1 + b + 1
        eta_optimal = (c2 - np.sqrt(c2**2 - 4 * c1 * c2)) / (2 * c1)
        sinr_optimal = ((1-eta_optimal)*a1*eta_optimal*a2) / ((1-eta_optimal)*a1*sigma2 + (eta_optimal*a2 + sig
        sinr_equal = ((1-0.5)*a1*0.5*a2) / ((1-0.5)*a1*sigma2 + (0.5*a2 + sigma2)*(b + sigma2))
        sinr_optimal_samples.append(sinr_optimal)
        sinr_equal_samples.append(sinr_equal)
    SE_optimal_sim.append(np.log2(1 + np.mean(sinr_optimal_samples)))
    SE_equal_sim.append(np.log2(1 + np.mean(sinr_equal_samples)))
```

```
# Plotagem dos resultados
plt.figure(figsize=(8, 6))
plt.plot(PdB_full, SE_optimal_analytical, label='Optimal AA Scheme (Analytical)', linestyle='-', color='red')
plt.plot(PdB_full, SE_equal_analytical, label='Equal AA Scheme (Analytical)', linestyle='-', color='blue')
plt.scatter(PdB_sim, SE_optimal_sim, color='red', s=70, label='Optimal AA Scheme (Simulation)', marker='o', fac
plt.scatter(PdB_sim, SE_equal_sim, color='blue', s=70, label='Equal AA Scheme (Simulation)', marker='d', faceco
plt.title('Spectral Efficiency of FD Relay System')
plt.xlabel('Transmit Power PT (dB)')
plt.ylabel('Spectral Efficiency (bps/Hz)')
plt.ylim(9.5, 12)
plt.xlim(0, 30)
plt.legend()
plt.grid(True)
plt.show()
```


Spectral Efficiency of FD Relay System
Optimal AA Scheme (Analytical)
Equal AA Scheme (Analytical)
Optimal AA Scheme (Simulation)
Equal AA Scheme (Simulation)
Spectral Efficiency (bps/Hz)
Transmit Power PT (dB)
12.0
11.5
11.0
10.5
10.0
9.5
0
5
10
15
20
25
30